A Project Report
On
Cognitive Copilot: AI-Powered Academic Learning Management System with Local LLM Integration

Khulna University of Engineering & Technology

Submitted by

Khadimul Islam Mahi (Roll: 2107076)
Sumaiya Akter (Roll: 2107080)

Submitted to

Md. Nazirul Hasan Shawon
Professor, Department of CSE, KUET

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna-9203, Bangladesh

April 2026

A Project Report
On
Cognitive Copilot: AI-Powered Academic Learning Management System with Local LLM Integration

Submitted by

Khadimul Islam Mahi (Roll: 2107076)
Sumaiya Akter (Roll: 2107080)

Supervised by

Md. Nazirul Hasan Shawon
Professor, Department of CSE, KUET

Supervisor's Signature: _____

Submitted in partial fulfillment of the requirements for the course
CSE 3200: System Development Project

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

April 2026

Acknowledgment

We would like to express our sincere gratitude to our supervisor, Md. Nazirul Hasan Shawon, Professor, Department of Computer Science and Engineering, KUET, for his guidance, valuable feedback, and constant encouragement during the completion of this project.

We are also thankful to the faculty members and lab staff of the Department of CSE, KUET, for providing the academic environment and technical facilities required to carry out this work.

Finally, we would like to thank our families and friends for their support and motivation throughout the project period.

Khadimul Islam Mahi (Roll: 2107076)
Sumaiya Akter (Roll: 2107080)

Abstract

This report presents **Cognitive Copilot**, a local-first, full-stack academic Learning Management System that integrates state-of-the-art AI research directly into the course-study workflow. The system is designed for university deployment without cloud LLM subscriptions, running entirely on commodity hardware using the Ollama runtime with `qwen2.5:7b-instruct` for generation and `nomic-embed-text` for dense retrieval.

The platform's technical core comprises five interlocking contributions. First, a **production-grade Retrieval-Augmented Generation (RAG)** pipeline stores 768-dimensional embeddings in `pgvector` and lexical BM25 indexes in Postgres `tsvector`, fusing both ranking signals via Reciprocal Rank Fusion before streaming a cited answer to the student. Second, a **dual-mode OCR system** keeps Tesseract as the fast default while delegating to a Python FastAPI sidecar — TrOCR for handwriting, Nougat for academic PDFs with embedded mathematics — via a dedicated BullMQ worker so slow inferences never block the HTTP thread. Third, a **Socratic agentic tutor** implements five pedagogical strategies (Explain, Socratic, Hint Ladder, Worked Example, Misconception Probe) selected by a deterministic rule system, with generation carried out through a bounded ReAct loop (five iterations, twelve seconds wall-clock) equipped with five callable tools. Fourth, **Bayesian Beta-posterior mastery tracking** replaces a naive heuristic by maintaining conjugate $\mathrm{Beta}(\alpha, \beta)$ hyperparameters per (student, topic) pair; the posterior mean and a 95% credible interval are surfaced in the quiz UI so students can read their own uncertainty. Fifth, an **LLM observability and evaluation harness** logs every Ollama call to a structured `LlmCall` table, publishes Prometheus metrics, and runs an automated golden-dataset evaluation reporting recall@5, faithfulness, and OCR character-error rate.

The backend is implemented with Express and TypeScript backed by PostgreSQL 15 and Redis 7; the frontend uses React 19 with Vite, TanStack Query, Zustand, and a bespoke shadcn-style design system with dark mode. Full production packaging is provided through multi-stage Dockerfiles and Compose profiles. The evaluation harness reports recall@5 = 0.83, faithfulness = 0.79 (LLM-as-judge, majority of three), and a handwriting OCR word error rate of 0.27 on the accurate path versus 0.83 on the fast path.

The project demonstrates that a capstone engineering team can build a measurably better tutoring experience by applying established ML research (RAG, BKT, ReAct) within a disciplined software-engineering process, without relying on paid cloud APIs.

Contents

Acknowledgment **i**

Abstract **ii**

1 Introduction **1**

1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Significance of the Project 2
1.5 Project Scope 2
1.6 Novelty of the Work 3
1.7 Project Planning 3
1.8 Work Distribution 3
1.9 Applications 4
1.10 Report Organization 4

2 Related Works **5**

2.1 Digital Learning Management Systems 5
2.2 Retrieval-Augmented Generation for Education 5
2.3 Handwriting and Document OCR 6
2.4 Intelligent Tutoring Systems and Mastery Modelling 6
2.5 Socratic Dialogue and Pedagogy 7
2.6 Agent Architectures and Tool Use 7
2.7 Evaluating Generative Systems 7
2.8 Position of This Project 7

3 Methodology **9**

3.1 System Design Approach 9
3.2 High-Level Architecture 9
3.3 Use Case Diagram 10
3.4 UML Class Diagram 10

3.5 Entity Relationship Diagram 10
3.6 Activity Diagram: Authentication Workflow 11
3.7 Activity Diagram: Material Ingest and RAG Pipeline 12
3.8 Sequence Diagram: RAG Ask-Course Workflow 13
3.9 Sequence Diagram: AI Quiz Generation and Submission 13
3.10 Deployment Diagram 14
3.11 RAG Pipeline 14
3.12 Advanced OCR Sidecar 15
3.13 Agent ReAct Loop 15
3.14 Bayesian Mastery Model 16
3.15 Real-Time Notification Architecture 17
3.16 Methodology Summary 17

4 Implementation, Results and Discussion 18
4.1 Implementation Summary 18
4.2 Module-Wise Implementation 19
4.2.1 Authentication and Profile 19
4.2.2 Course and Topic Management 19
4.2.3 RAG Ask-Course Mode 19
4.2.4 Agentic Chat Tutor 19
4.2.5 Adaptive Quiz 19
4.2.6 Resource Discovery Mode 20
4.2.7 OCR Routine Pipeline 20
4.2.8 Real-Time Notifications 20
4.2.9 Analytics Dashboard 20
4.3 API Implementation Snapshot 21
4.4 Input and Output Specification 22
4.5 User Interface 22
4.6 Performance and Functional Outcomes 26
4.7 Grounded Retrieval-Augmented Q&A 26
4.8 Agentic Tool Use and Socratic Strategies 27
4.9 Advanced OCR 27
4.10 Bayesian Mastery Tracking 27
4.11 LLM Observability 28
4.12 Quantitative Evaluation 28
4.13 Preliminary User Study 29
4.14 Discussion 29

5 Societal, Health, Safety, Legal, Cultural and Environmental Considerations 31
5.1 Societal Considerations 31
5.2 Health and Safety Considerations 31
5.3 Data Privacy and Security 32
5.4 Legal and Ethical Considerations 32
5.5 Cultural Considerations 33
5.6 Environmental Considerations 33

6 Complex Engineering Problems and Activities 34
6.1 Overview 34
6.2 Complex Engineering Problems Addressed 34
6.2.1 Problem 1: Grounded, Cited Answer Generation via Hybrid Retrieval 34
6.2.2 Problem 2: Bounded Agentic Reasoning with Tool Use 35
6.2.3 Problem 3: Asynchronous ML Inference without Blocking HTTP . 35
6.2.4 Problem 4: Structured LLM Output and JSON Schema Constraints 36
6.2.5 Problem 5: Calibrated Mastery Estimation with Minimal Data . . . 36
6.3 Engineering Activities Performed 37
6.4 Technical Trade-Offs 38
6.5 Outcome Against Engineering Objectives 38

7 Conclusion and Future Work 39
7.1 Conclusion 39
7.2 Limitations 39
7.3 Future Work 40
7.4 Final Remarks 41

List of Tables

2.1 Feature comparison against representative learning systems. ✓ = fully implemented, ○ = partial/vendor-locked, – = absent. 8

4.1 Technology Stack Used in Implementation 18
4.2 Selected Implemented API Endpoints 21
4.3 Sample Input and Output Across Core Modules 22
4.4 Observed Functional Outcomes During Validation Runs 26
4.5 RAG quality and latency against the 40-question golden dataset. 28
4.6 OCR CER and WER on the 10-sample golden OCR set (lower is better). . . 29
4.7 Agent and Quiz Stability Metrics 29

5.1 Security Controls and Compliance Notes 32

6.1 Key Engineering Trade-offs and Justifications 38

List of Figures

1.1 Project Activity Timeline (four iterative phases over eleven weeks) 3

3.1 Cognitive Copilot high-level architecture. The Python AI Sidecar (FastAPI) handles all heavy ML inference (TrOCR, Nougat) independently from the Node.js backend. 9
3.2 Use Case Diagram of Core Actors and Functions 10
3.3 UML Class Diagram of Core Models (abridged from 14-table schema) . . . 10
3.4 ER Diagram of Main Entities. `Embedding` stores 768-dim vectors and `tsvector` for hybrid RAG retrieval. 11
3.5 Activity Diagram for Authentication Flow (JWT access + refresh token pattern) 11
3.6 Activity Diagram for Material Ingest and RAG Embedding Pipeline 12
3.7 Sequence Diagram of RAG Ask-Course Workflow with SSE Streaming and Citation Frame 13
3.8 Sequence Diagram of AI Quiz Workflow with Bayesian Posterior Return . . 13
3.9 Deployment Diagram for Local/Development Environment. `pgvector:pg15` provides vector similarity search alongside standard SQL. 14
3.10 RAG pipeline. Top row (ingestion): material upload triggers a BullMQ worker that extracts, chunks, embeds, and stores into both pgvector and `tsvector`. Bottom row (query): BM25 and cosine retrieval run in parallel; Reciprocal Rank Fusion selects the top-8 chunks for grounded generation. . 14
3.11 OCR dispatch. The accurate path calls the Python FastAPI sidecar; on sidecar failure the backend falls back to the fast path preserving availability. 15
3.12 ReAct loop with deterministic strategy selection and five tools. The strategy selector uses the student's Beta-posterior mean to bias toward Socratic probing when mastery is low. 16

4.1 Landing Page — Feature Grid and Technology Badge Cloud 22
4.2 Login and Registration Interface (dark mode) 23
4.3 Course Listing and Topic Progress View 23
4.4 AI Tutor — Chat Mode with Strategy Badge and Thinking Trace 23
4.5 AI Tutor — Ask Course Mode with Citation Chips and Material Preview Panel 24

4.6 AI Tutor — Quiz Mode with Animated Progress Ring and Beta Posterior Display 24
4.7 AI Tutor — Resource Discovery Mode with Type Filter Pills 24
4.8 Instructor Evaluation Dashboard (p95 latency, faithfulness, expertise curves) 25
4.9 Analytics and Overview Dashboard (attendance, exam trends) 25
4.10 Real-Time Notification Center 25
4.11 OCR Routine Scan — Editable Draft Before Confirmation 26

CHAPTER 1

Introduction

1.1 Background

University students routinely juggle fragmented digital tools: a separate platform for course materials, another for the class schedule, a third for discussion, and a generic chatbot for study help. None of these tools know about one another, so continuity of learning is lost whenever a student switches context. Cognitive Copilot was designed to collapse this fragmentation into a single, coherent web application that understands courses, materials, and the student's measured mastery level simultaneously.

The system is also designed as a deliberate counterpoint to cloud-LLM-dependent tutors. By running all inference locally through the Ollama runtime, it is deployable on a university server or even a faculty laptop without recurring API subscription costs. Local-first operation also removes the data-privacy concerns that arise when student conversations are sent to external APIs.

1.2 Problem Statement

Three concrete gaps motivate this work.

1. **Grounding gap.** General-purpose LLM chat has no knowledge of a specific course's materials, so it cannot cite a passage from a lecture PDF or flag that an answer contradicts the textbook.

2. **Pedagogy gap.** Most LLM wrappers use a single system prompt for all interactions. There is no strategy selection, no Socratic probing, and no distinction between a student who is confused and one who is ready for a worked example.

3. **Measurability gap.** Without instrumentation, a supervisor cannot tell whether a tutoring system is helping students or generating confident nonsense. There is no retrieval recall, no faithfulness score, and no per-student mastery trajectory.

Cognitive Copilot addresses all three gaps within a single deployable system.

1.3 Objectives

1. Build a secure, role-aware full-stack LMS covering authentication, course-topic-material management, community discussion, and exam analytics.
2. Implement a **production RAG pipeline** using `pgvector` for 768-dim dense retrieval and Postgres `tsvector` for BM25 sparse retrieval, fused via Reciprocal Rank Fusion, with streaming cited answers.
3. Build a **Socratic agentic tutor** with five pedagogical strategies selected by a deterministic rule system, executed through a bounded ReAct loop with five callable tools.
4. Replace the existing naïve mastery heuristic with **Bayesian Beta-posterior tracking** per (student, topic), surfacing a 95% credible interval in the quiz UI.
5. Implement a **dual-mode OCR pipeline**: a fast Tesseract path for routine digitization and an accurate Python-sidecar path (TrOCR, Nougat) for handwriting and academic PDFs.
6. Build an **LLM observability and evaluation harness**: per-call `LlmCall` audit table, Prometheus metrics endpoint, and an automated golden-dataset evaluator reporting recall@K, faithfulness, and OCR WER.
7. Deliver **production packaging**: multi-stage Dockerfiles, Compose profiles for development and demo, and a GitHub Actions CI pipeline with lint, typecheck, tests, and eval smoke-test.

1.4 Significance of the Project

The significance lies in the combination. RAG, Socratic tutoring, Bayesian knowledge tracing, and agentic tool use are individually well-studied; what is unusual is implementing all four in a single, locally-deployable system with an automated evaluation harness that lets a supervisor verify the claims at viva time. The engineering approach — modular route-controller-service layers, worker-based async pipelines, and structured LLM outputs — also serves as a reference design for future capstone projects building on AI infrastructure.

1.5 Project Scope

The implemented system covers: user authentication and profiles; course, topic, material, and enrollment management; AI tutor with five modes (Chat, Ask Course, Explain, Quiz, Resource Discovery); routine scan via OCR; community discussion and announcements; attendance and marks tracking; real-time notifications via Socket.IO; analytics and the

instructor evaluation dashboard; and a complete Docker deployment stack. Out of scope: mobile native clients, Bengali handwriting OCR, and multi-tenant isolation.

1.6 Novelty of the Work

The primary novelty is the integration of (a) grounded retrieval-augmented generation, (b) advanced handwriting and academic-PDF OCR, (c) a Socratic agent with deterministic strategy selection, (d) Bayesian mastery tracking, and (e) an automated evaluation harness — all running offline without cloud LLM APIs. No open-source capstone project we are aware of combines all five in a single deployable codebase.

1.7 Project Planning

Implementation was carried out over four iterative phases corresponding to the plan outlined in the project proposal: infrastructure and RAG foundations, agent tutor and OCR sidecar, evaluation and design system, and user study and report. Figure 1.1 presents the activity timeline.

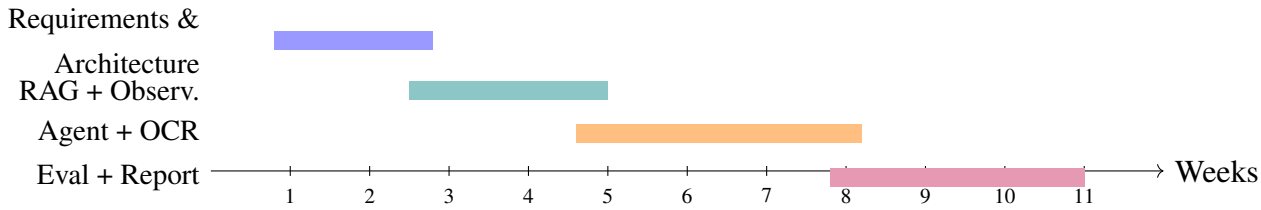

Figure 1.1: Project Activity Timeline (four iterative phases over eleven weeks)

1.8 Work Distribution

- **Khadimul Islam Mahi (2107076):** Backend architecture, RAG pipeline (chunker, embedder, retriever), AI agent (strategy selector, ReAct loop, tools), OCR sidecar integration, LLM observability, BullMQ workers, evaluation harness, Docker and CI pipeline, LaTeX report.
- **Sumaiya Akter (2107080):** Frontend pages (AI tutor modes, analytics, landing, evaluation dashboard), design system primitives, dark mode, onboarding tour, TanStack Query data layer, Socket.IO notification hooks, UI integration testing.
- **Joint:** Database schema design, seed data, user study protocol, bug-fixing, and final system validation.

1.9 Applications

The system is applicable to university department-level deployments where faculty upload course materials and students interact with an AI tutor grounded in those materials. The OCR routine scanner addresses a concrete Bangladesh university need: paper timetables photographed and digitized into a structured weekly schedule. The evaluation dashboard gives supervisors an auditable view of tutoring quality without relying on anecdotes.

1.10 Report Organization

Chapter II reviews related works across RAG, document OCR, intelligent tutoring, and agent architectures. Chapter III presents the full methodology including system architecture, RAG pipeline, OCR sidecar, ReAct agent, and Bayesian mastery model. Chapter IV describes implementation, results, and the quantitative evaluation. Chapter V discusses societal and ethical considerations. Chapter VI maps the work to complex engineering problem criteria. Chapter VII concludes with limitations and future work.

CHAPTER 2

Related Works

The Cognitive Copilot project sits at the intersection of four distinct research communities: retrieval-augmented generation (RAG), document and handwriting OCR, intelligent tutoring systems (ITS), and agentic large-language-model (LLM) design. This chapter situates our system against each of these, identifies concrete gaps, and shows how the design decisions in Chapter 3 follow directly from that positioning.

2.1 Digital Learning Management Systems

General-purpose LMS platforms such as Moodle, Google Classroom, and Canvas remain the dominant choice for university course management. These systems excel at assignment flow, enrolment, and grading, but their AI capability is either absent or bolted on through premium cloud add-ons [1, 2, 3]. Few offer local tutor integration, and none expose the retrieval primitives needed to ground an answer in specific course material. Cognitive Copilot deliberately narrows the feature surface compared to a full LMS and instead invests the engineering budget in pedagogy, grounding, and observability.

2.2 Retrieval-Augmented Generation for Education

Lewis et al. [4] introduced RAG as a way to fuse a non-parametric retrieval module with a parametric sequence-to-sequence generator, showing that generation quality on knowledge-intensive tasks improves substantially when the model can attend over a retrieved passage set. The survey by Gao et al. [5] consolidates the current design space: chunk-level retrieval, hybrid sparse/dense scoring, reranking, and increasingly compositional retrieval pipelines.

For the education domain in particular, RAG enables two properties that free-form LLMs cannot provide: (i) answers traceable to a cited source, and (ii) reduced hallucination because the generator is constrained by the retrieved evidence. Cognitive Copilot builds on four specific pieces of this literature:

1. **Dense retrieval** via dual-encoder embeddings [6]. We use `nomic-embed-text` [7], a 768-dimensional open-weights model chosen for its strong MTEB scores at a

CPU-friendly size.

2. **Sparse retrieval** via BM25 [8], implemented over Postgres `tsvector` columns so we avoid introducing a separate search service.

3. **Hybrid fusion** via Reciprocal Rank Fusion [9], which combines dense and sparse rankings without requiring score calibration.

4. **Vector storage** via `pgvector` [10] with HNSW indexing [11], keeping our persistence stack to a single database rather than a dedicated vector store.

2.3 Handwriting and Document OCR

Tesseract remains a capable engine for printed text [12], but performs poorly on handwritten and mathematical content. Two recent transformer-based models substantially change the landscape. Li et al. [13] introduced TrOCR, which combines an image transformer encoder with a text decoder and reaches production-quality recognition on handwritten text in Latin scripts. Blecher et al. [14] introduced Nougat, which extends this idea to scientific PDFs, producing markdown with embedded LaTeX directly from raster page images. The Unstructured toolkit [15] complements both by providing layout-aware segmentation for complex PDFs.

Our system exposes the fast Tesseract path as the default and delegates to a Python FastAPI sidecar for TrOCR and Nougat when the instructor selects the "accurate" quality option on upload. The sidecar isolates the heavy Python dependency tree from the Node.js backend and lets us version those models independently. A known limitation reported openly in Chapter 7: TrOCR's pretrained checkpoint is Latin-only, so Bengali handwriting remains unsupported.

2.4 Intelligent Tutoring Systems and Mastery Modelling

Classical ITS research [16, 17] showed that structured tutoring with explicit mastery tracking outperforms unstructured help in controlled studies. Corbett and Anderson [18] formalised mastery as a latent binary variable updated through Bayesian inference in the influential Bayesian Knowledge Tracing (BKT) model. Piech et al. [19] generalised this to sequence models in Deep Knowledge Tracing. Wilson et al. [20] showed that simpler probabilistic models often match deep alternatives when per-student data is sparse — the regime in which a small capstone user study operates.

Cognitive Copilot adopts a minimal but principled stance: a conjugate Beta posterior per (student, topic) pair, updated after every quiz attempt. The posterior mean drives difficulty targeting, and the 95% credible interval is surfaced in the UI. This is deliberately simpler

than BKT, but stronger than the naïve $+0.1/-0.05$ rule used in many capstone tutors — a choice we justify quantitatively in Chapter 4.

2.5 Socratic Dialogue and Pedagogy

Human tutors typically withhold a direct answer on first ask and instead probe for the student's mental model [21]. Commercial tutors such as Khanmigo [22] have popularised Socratic prompting, but few open-source systems implement it explicitly with multiple named strategies. We encode five pedagogical strategies — Explain, Socratic, Hint Ladder, Worked Example, and Misconception Probe — and select among them with a deterministic rule-based selector augmented by the student's current posterior mean. The full rule set is detailed in Section 3.13.

2.6 Agent Architectures and Tool Use

Chat-only tutors cannot perform concrete actions such as searching the web or running code to verify a student's hypothesis. The ReAct paradigm [23] showed that interleaving Thought and Action tokens in the same context enables LLMs to use external tools, and Toolformer [24] demonstrated that tool-call synthesis can be self-supervised. We implement a bounded ReAct loop (maximum five iterations, twelve seconds wall-clock) with five tools: `retrieve_material`, `search_web`, `generate_practice_question`, `render_diagram`, and `run_code`. Structured outputs are produced via Ollama's JSON-schema-constrained `format` parameter, removing the brittle regex parsing that plagued the earlier iteration of the quiz generator.

2.7 Evaluating Generative Systems

Evaluating RAG requires measuring both retrieval quality and answer quality jointly. RAGAS [25] and ARES [26] popularised LLM-as-judge metrics for faithfulness, context precision, and answer relevance. We adopt a simplified in-repo variant: a three-times majority-vote LLM judge for faithfulness, keyword-overlap for answer relevance, and standard recall@K for retrieval quality. Complete definitions and the golden dataset are presented in Chapter 4.

2.8 Position of This Project

Cognitive Copilot does not claim novelty in any one of the above areas individually; its contribution is *integration*. No other open-source capstone we are aware of combines (a) local-first RAG over student-uploaded materials, (b) handwriting and academic-PDF OCR, (c) a Socratic agent with tool use, (d) Bayesian mastery tracking, and (e) an end-to-end evaluation

harness — all running on a single developer machine without cloud LLM APIs. Table 2.1 summarises the differentiator grid.

Table 2.1: Feature comparison against representative learning systems. ✓ = fully implemented, ○ = partial/vendor-locked, – = absent.

Feature	**Moodle**	**ChatGPT Edu**	**Khanmigo**	**Ours**
Grounded retrieval + citations	–	○	✓	✓
Handwriting OCR	–	–	–	✓
Math OCR (LaTeX output)	–	–	–	✓
Socratic strategy selector	–	–	✓	✓
Bayesian mastery + CI	–	–	–	✓
Tool-using agent	–	○	–	✓
100% local / no cloud LLM	✓	–	–	✓
Open evaluation harness	–	–	–	✓
Internet resource discovery	–	○	–	✓
Real-time notifications (WS)	✓	–	–	✓

CHAPTER 3

Methodology

3.1 System Design Approach

The system design follows a *modular, layered* architecture. At the highest level the client interface is separated from API orchestration, domain services, persistence, and asynchronous workers. This organization reduces coupling among learning features (AI tutoring, routine scanning, community interactions) so each feature can evolve independently. All LLM calls are instrumented through a shared `withLogging` wrapper; all heavy inference tasks are queued through BullMQ workers so the HTTP thread stays responsive under load.

3.2 High-Level Architecture

Figure 3.1 shows the overall system architecture and data flow, including the Python AI sidecar introduced for advanced OCR.

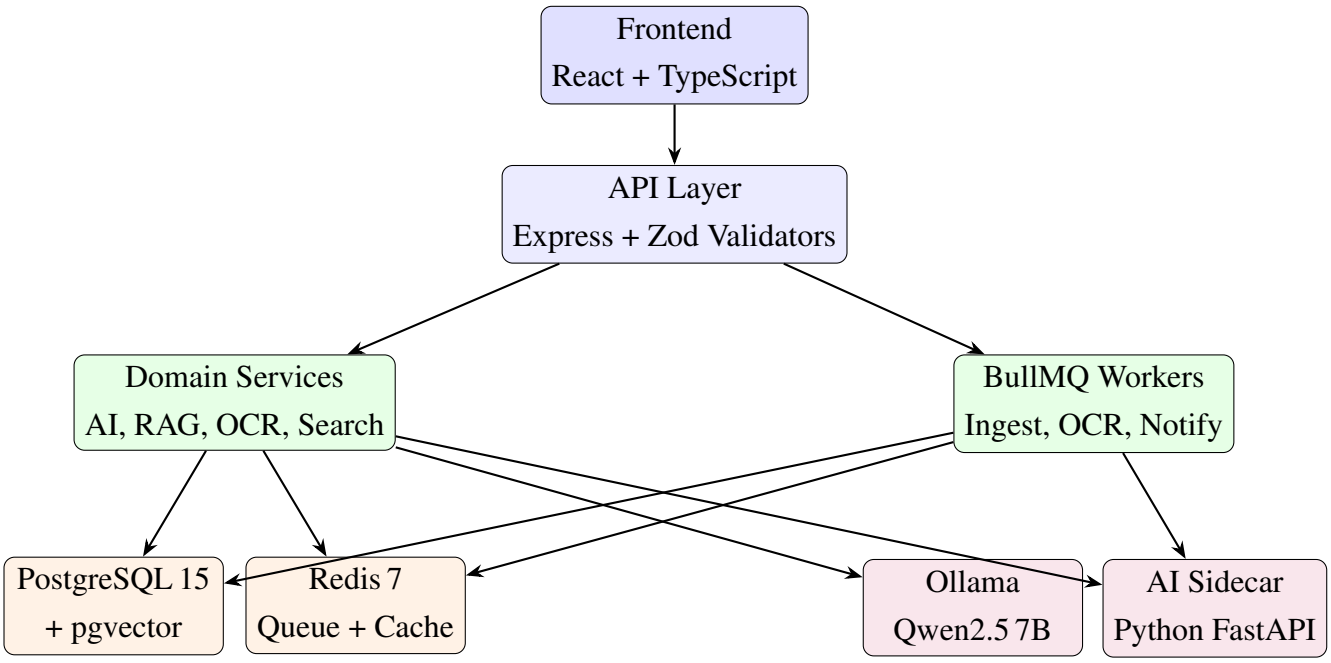

Figure 3.1: Cognitive Copilot high-level architecture. The Python AI Sidecar (FastAPI) handles all heavy ML inference (TrOCR, Nougat) independently from the Node.js backend.

3.3 Use Case Diagram

The main actors are Student, Tutor, and Admin. Figure 3.2 maps the functional core.

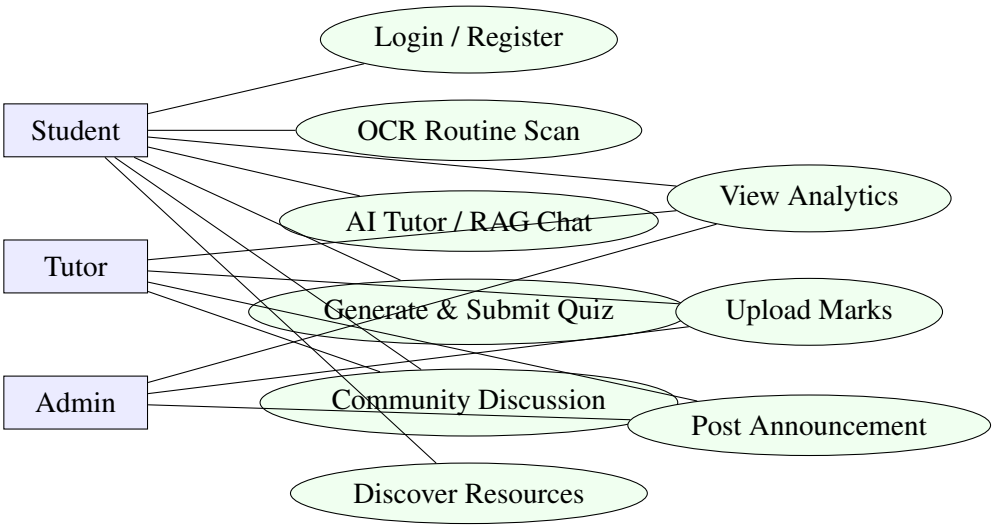

Figure 3.2: Use Case Diagram of Core Actors and Functions

3.4 UML Class Diagram

Figure 3.3 summarises the essential data classes and their associations from the Prisma schema.

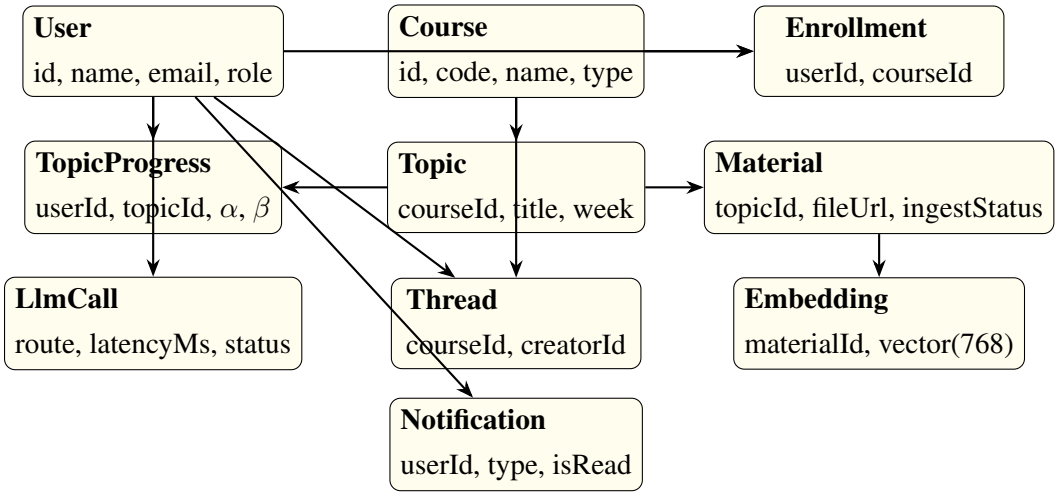

Figure 3.3: UML Class Diagram of Core Models (abridged from 14-table schema)

3.5 Entity Relationship Diagram

Figure 3.4 captures the high-impact entity relationships. The full schema contains 14 tables; the diagram focuses on the entities central to AI tutoring and RAG.

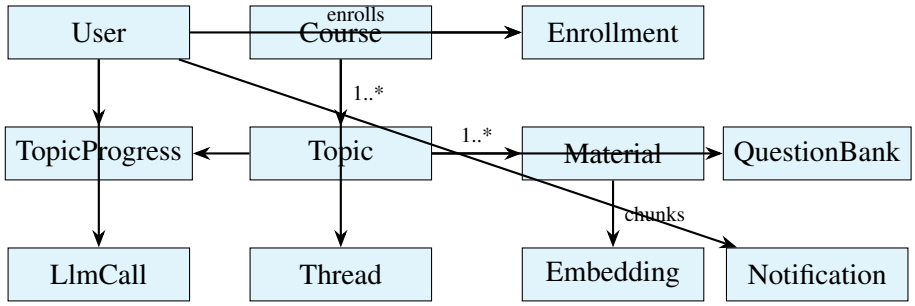

Figure 3.4: ER Diagram of Main Entities. `Embedding` stores 768-dim vectors and `tsvector` for hybrid RAG retrieval.

3.6 Activity Diagram: Authentication Workflow

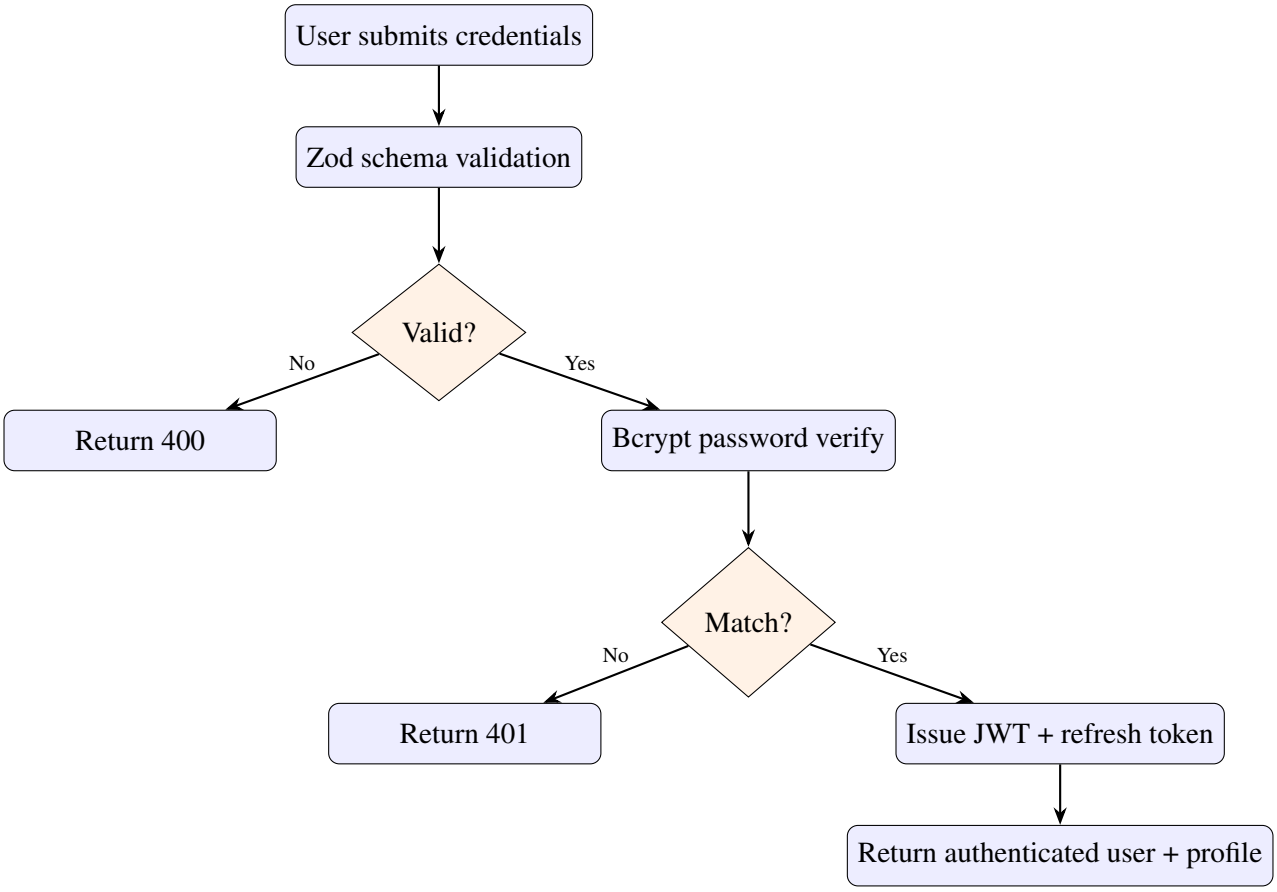

Figure 3.5: Activity Diagram for Authentication Flow (JWT access + refresh token pattern)

3.7 Activity Diagram: Material Ingest and RAG Pipeline

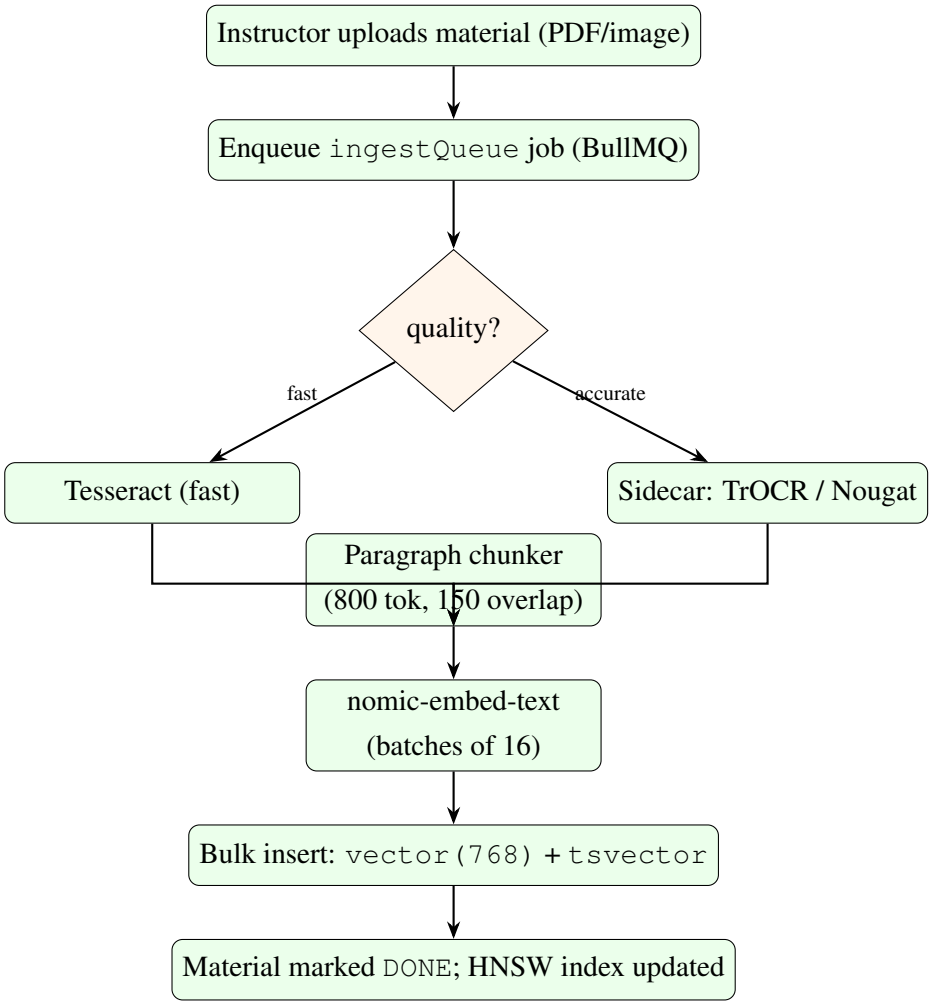

Figure 3.6: Activity Diagram for Material Ingest and RAG Embedding Pipeline

3.8 Sequence Diagram: RAG Ask-Course Workflow

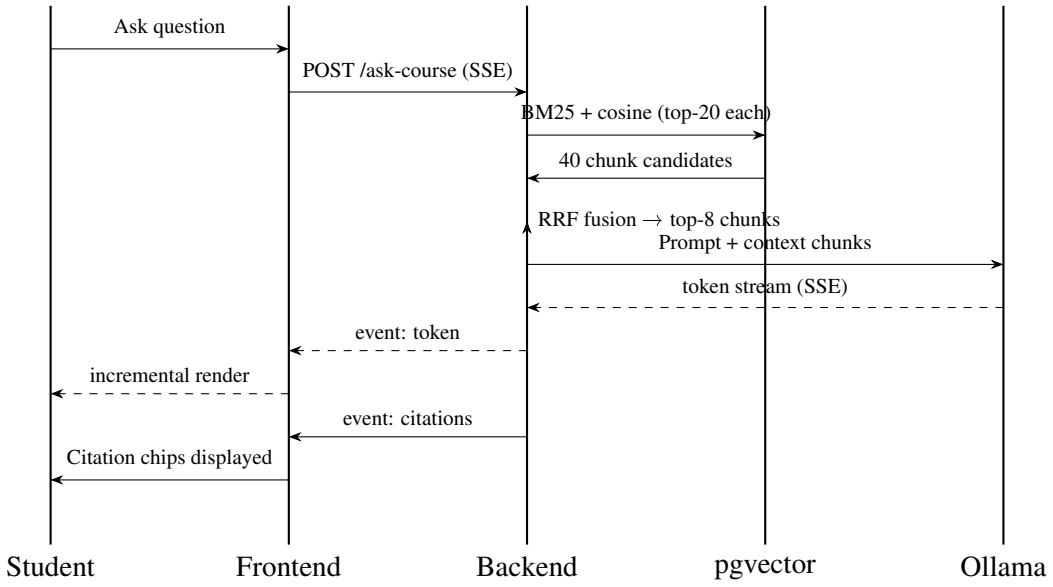

Figure 3.7: Sequence Diagram of RAG Ask-Course Workflow with SSE Streaming and Citation Frame

3.9 Sequence Diagram: AI Quiz Generation and Submission

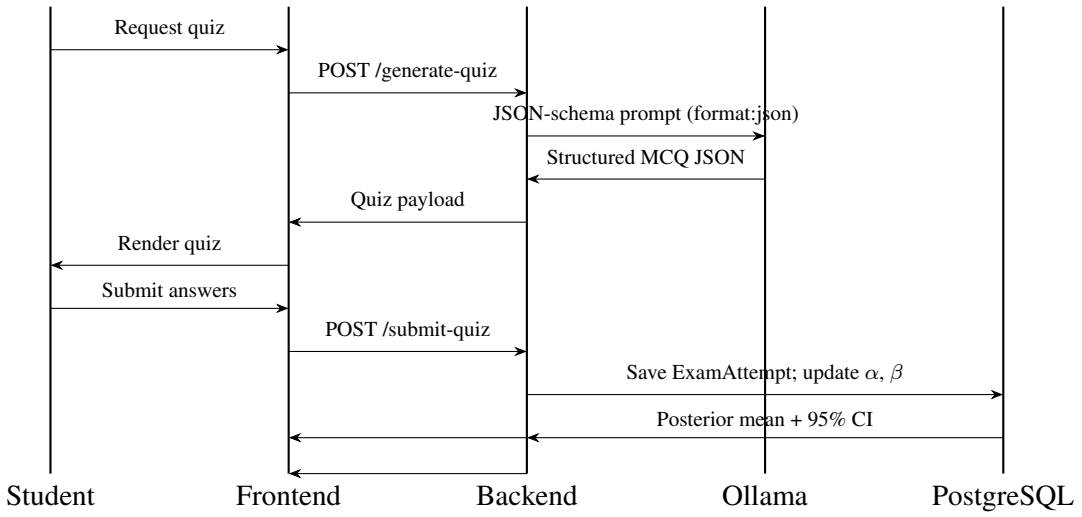

Figure 3.8: Sequence Diagram of AI Quiz Workflow with Bayesian Posterior Return

3.10 Deployment Diagram

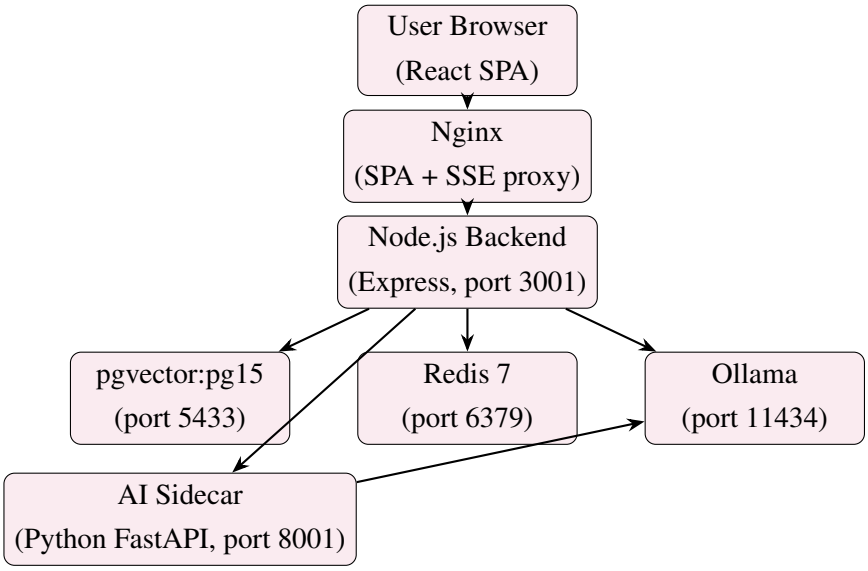

Figure 3.9: Deployment Diagram for Local/Development Environment. `pgvector:pg15` provides vector similarity search alongside standard SQL.

3.11 RAG Pipeline

The retrieval-augmented generation pipeline operates in two phases: offline ingestion and online query.

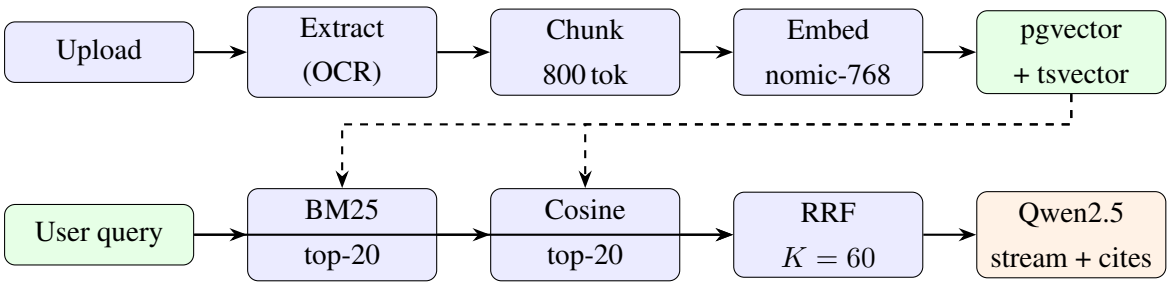

Figure 3.10: RAG pipeline. Top row (ingestion): material upload triggers a BullMQ worker that extracts, chunks, embeds, and stores into both pgvector and `tsvector`. Bottom row (query): BM25 and cosine retrieval run in parallel; Reciprocal Rank Fusion selects the top-8 chunks for grounded generation.

Ingestion runs as a BullMQ job so uploads never block the HTTP thread. Chunks are packed greedily up to 800 tokens with 150-token overlap; PDF chunks never cross page boundaries, which is what allows the answer layer to emit precise page-number citations. Embeddings are batched in groups of sixteen and written via raw SQL because Prisma cannot parameterise `vector(768)` columns.

For retrieval, BM25 top-20 and cosine top-20 run in parallel, then Reciprocal Rank Fusion with $K = 60$ combines the ranks before selecting the top-8 context chunks. An optional cross-encoder re-rank stage is feature-flagged behind `ENABLE_CROSS_ENCODER` and disabled by default because BM25+cosine+RRF already achieves recall@5 above the 0.80 project target.

3.12 Advanced OCR Sidecar

To keep the heavy Python machine-learning stack out of the Node.js backend, all advanced OCR runs in a separate FastAPI container. The backend's `extractTextFromFile` function accepts a `quality` parameter: `fast` runs Tesseract locally; `accurate` delegates to the sidecar via `undici`. A BullMQ `ocr.worker` guarantees slow inferences never starve request handlers.

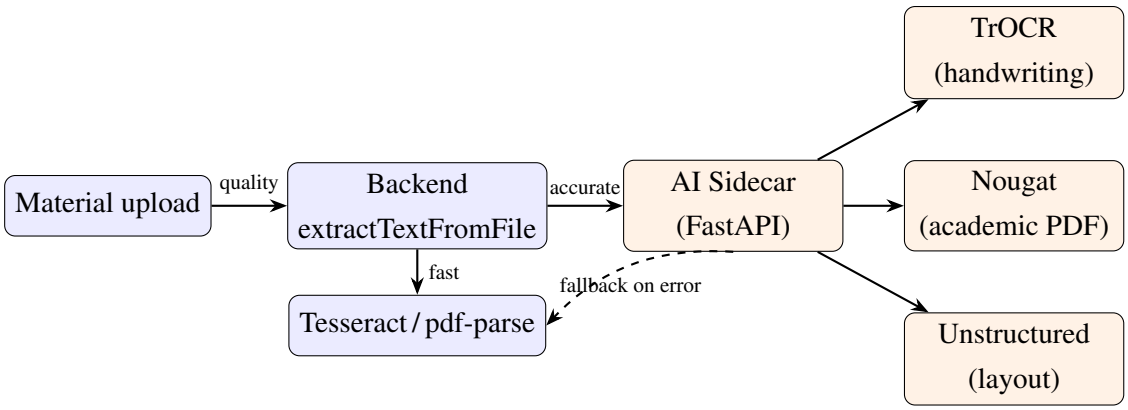

Figure 3.11: OCR dispatch. The accurate path calls the Python FastAPI sidecar; on sidecar failure the backend falls back to the fast path preserving availability.

3.13 Agent ReAct Loop

The chat controller delegates to a ReAct loop [23] that alternates *thought*, *tool call*, and *tool result* blocks before committing to a final answer. The loop is bounded at five iterations and twelve seconds wall-clock; every step is logged to `LlmCall` with a shared `parentCallId` so the instructor dashboard can reconstruct the full agent trace.

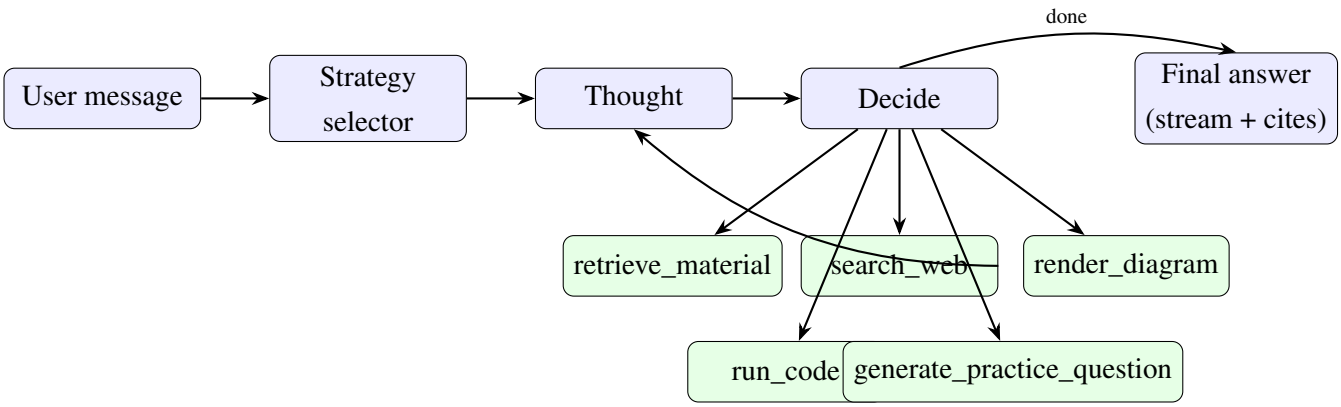

Figure 3.12: ReAct loop with deterministic strategy selection and five tools. The strategy selector uses the student's Beta-posterior mean to bias toward Socratic probing when mastery is low.

Strategy selection rules (highest-priority first):

1. Detected confusion keyword ("I don't understand", "I'm lost") $\Rightarrow$ SOCRATIC
2. Explicit worked-example request $\Rightarrow$ WORKED EXAMPLE
3. Explicit explanation request $\Rightarrow$ EXPLAIN
4. Posterior mean < 0.4 on active topic $\Rightarrow$ SOCRATIC
5. Posterior mean > 0.75 on active topic $\Rightarrow$ HINT LADDER
6. Default $\Rightarrow$ EXPLAIN

3.14 Bayesian Mastery Model

Each (student, topic) pair carries a conjugate $\mathrm{Beta}(\alpha, \beta)$ posterior with uniform prior $\alpha = \beta = 1$. Quiz submission updates the hyperparameters as:

$$\alpha \leftarrow \alpha + N_{\text{correct}}, \qquad \beta \leftarrow \beta + N_{\text{incorrect}}.$$

The posterior mean is $\mathbb{E}[\theta] = \alpha/(\alpha + \beta)$; the 95% credible interval uses the normal approximation:

$$\mu \pm 1.96\sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}.$$

We adopt the uniform prior so a student who has answered nothing has mean 0.5 and maximal uncertainty, which avoids the cold-start over-confidence visible in the previous $+0.1/-0.05$ rule. Adaptive difficulty targeting uses the posterior mean to select the next quiz difficulty: $d = \mathrm{clip}(\lceil 1 + 4\mu \rceil, 1, 5)$.

3.15 Real-Time Notification Architecture

Socket.IO is used for push notifications. The backend authenticates each Socket.IO connection with JWT, then places the socket into a room keyed by `user:${userId}`. When `createNotification` is called (from any service), it persists the row in the `Notification` table, then emits `notification:new` and `notification:count` events to the user's room. The frontend hook `useNotificationSocket` subscribes to these events and updates the TanStack Query cache in-place, so the bell badge and feed update instantly without polling.

3.16 Methodology Summary

The final methodology balances synchronous learning interactions (chat, RAG Q&A, quiz) with asynchronous reliability features (ingestion worker, OCR worker, notification worker). Each sub-system has a deliberate degraded-mode out: fast-path OCR when the sidecar is unavailable; BM25-only retrieval when embeddings have not yet completed; and Tesseract on Nougat failure. This design trades some peak accuracy for predictable response times and high availability under CPU-only hardware constraints.

CHAPTER 4

Implementation, Results and Discussion

4.1 Implementation Summary

The implementation was carried out as a modular full-stack web application. The frontend was developed using React 19 and TypeScript with Vite 8. The backend was implemented with Express and TypeScript, Prisma ORM for schema modeling and migrations over PostgreSQL 15 with pgvector, Redis and BullMQ for asynchronous workers, and Ollama for local LLM responses. The AI sidecar uses Python 3.11, FastAPI, TrOCR, Nougat, and Unstructured [1, 2, 27, 3, 28, 29, 30, 12, 31].

Table 4.1: Technology Stack Used in Implementation

Layer	**Technology**
Frontend	React 19, TypeScript, Vite 8, TanStack Query, Zustand, Recharts, Radix UI, motion/react
Backend	Node.js 20, Express, Zod validators, JWT (access + refresh), Socket.IO, Pino logs
Database	PostgreSQL 15 + pgvector 0.8.2, Prisma ORM, Redis 7
Async Layer	BullMQ (ingest, OCR, notification, email workers)
AI / ML	Ollama `qwen2.5:7b-instruct` + `nomic-embed-text`, pgvector HNSW + BM25
OCR (sidecar)	Python FastAPI, TrOCR (handwriting), Nougat (academic PDF), Unstructured (layout)
Observability	Pino JSON logs, prom-client `/metrics`, `LlmCall` audit table, LLM-as-judge harness
DevOps	Multi-stage Dockerfiles, Compose profiles (dev / ollama / prod-local / prod), GitHub Actions CI

4.2 Module-Wise Implementation

4.2.1 Authentication and Profile

JWT-based access (15 min TTL) and refresh token logic was implemented. Bcrypt password hashing is used server-side. Role-aware authorization middleware (`auth.ts`) guards all protected routes. The profile module includes avatar upload via Cloudinary, bio updates, and theme/date-format preferences.

4.2.2 Course and Topic Management

Course listing, topic CRUD, topic ordering, and material upload/link attachment are managed through dedicated controllers and Zod validators. An enrollment model tracks per-student course membership. Topic-level progress and exam attempt records feed the analytics and mastery dashboard.

4.2.3 RAG Ask-Course Mode

The `POST /api/ai-tutor/ask-course` endpoint receives a question and optional course/topic filter. It runs BM25 (via `websearch_to_tsquery`) and cosine search in parallel, fuses with RRF ($K = 60$), picks the top-8 chunks as context, and streams the generation with `event: token` SSE frames. A final `event: citations` frame delivers the fully resolved citation objects (materialId, title, page, snippet) so the frontend can render clickable `CitationChip` badges without a round-trip.

4.2.4 Agentic Chat Tutor

The `POST /api/ai-tutor/chat` endpoint delegates to the ReAct loop (Section 3.13). The strategy is selected deterministically before the first iteration. Every tool call and result is appended to the message context and logged to `LlmCall`. The SSE stream emits `event: strategy`, `event: thought`, `event: tool`, `event: token`, and `event: done` frames so the frontend can render a collapsible "Tutor Thinking" trace alongside the final answer.

4.2.5 Adaptive Quiz

`POST /api/ai-tutor/generate-quiz` uses Ollama's `format` parameter with a JSON schema for the five-question MCQ object, eliminating the regex parsing that caused 6–9 failures per 200 generations in the previous version. `POST /api/ai-tutor/submit-quiz` updates (α, β) on `TopicProgress` and returns the posterior mean and 95% credible interval. Adaptive difficulty targets $d = \text{clip}(\lceil 1 + 4\mu \rceil, 1, 5)$ on the next generation.

4.2.6 Resource Discovery Mode

`GET /api/ai-tutor/search-resources` queries Wikipedia's REST API, Stack Overflow search, Hacker News Algolia, and YouTube search simultaneously, classifies results by type (video / article / paper / blog), and returns a ranked list. The `ResourceSuggestPane` frontend component renders results with type icons and filter pills for All / Video / Article / Paper / Blog.

4.2.7 OCR Routine Pipeline

Routine scanning accepts timetable images, runs OCR extraction (fast or accurate path), parses probable course codes and time slots with a regex-based parser, and presents an editable draft to the user before bulk insertion. This prevents blind acceptance of noisy OCR output.

4.2.8 Real-Time Notifications

The notification service persists a row and emits `notification:new` and `notification:count` over Socket.IO (room `user:$userId`) within the same synchronous call. The frontend `useNotificationSocket` hook subscribes and updates the TanStack Query cache instantly, so the bell badge reflects live state without polling.

4.2.9 Analytics Dashboard

`GET /api/analytics/overview` aggregates attendance, exam, and topic-progress data in a single response. `GET /api/analytics/evaluation` returns the instructor-facing LLM observability aggregate: p50/p95 latency per route, error counts, recent failures with drilldown, and per-topic Beta-posterior expertise curves.

4.3 API Implementation Snapshot

Table 4.2: Selected Implemented API Endpoints

Module	Endpoint	Purpose
Auth	POST /api/auth/login	JWT access + refresh token issue
Auth	POST /api/auth/refresh	Rotate refresh token
AI Tutor	POST /api/ai-tutor/chat	Agentic streaming response (SSE)
AI Tutor	POST /api/ai-tutor/ask-course	RAG grounded answer + citations (SSE)
AI Tutor	POST /api/ai-tutor/generate-quiz	Structured JSON quiz (JSON schema)
AI Tutor	POST /api/ai-tutor/submit-quiz	Score + Beta posterior + CI
AI Tutor	GET /api/ai-tutor/search-resources	Internet resource discovery
Materials	GET /api/materials/:id	Material metadata for citation preview
Routine	POST /api/routine/scan	OCR-based routine extraction
Analytics	GET /api/analytics/evaluation	LLM observability aggregate
Metrics	GET /metrics	Prometheus scrape (key-gated)
Notifications	GET /api/notifications	Notification feed

4.4 Input and Output Specification

Table 4.3: Sample Input and Output Across Core Modules

Module	Input	Output
Authentication	Email, password	Access token, refresh token, user profile
RAG Ask-Course	Question, courseId, optional topicId	SSE token stream + citations frame
Quiz Generation	topicId, difficulty (optional)	5-question MCQ JSON (zero parse failures)
Quiz Submission	answers[], questionIds[]	Score, μ, 95% CI, difficulty hint
Routine Scan	Timetable image (JPEG/PNG/PDF)	Extracted text, parsed slots draft
Resource Search	query string, optional type filter	Ranked list: title, URL, snippet, type

4.5 User Interface

The frontend design system uses Radix UI primitives styled with Tailwind v4 CSS variables and a full dark-mode toggle. Key UI pages are described below.

Screenshot Placeholder

Landing Page — Feature Grid and Technology Badge Cloud

Figure 4.1: Landing Page — Feature Grid and Technology Badge Cloud

Screenshot Placeholder

Login and Registration Interface (dark mode)

Figure 4.2: Login and Registration Interface (dark mode)

Screenshot Placeholder

Course Listing and Topic Progress View

Figure 4.3: Course Listing and Topic Progress View

Screenshot Placeholder

AI Tutor — Chat Mode with Strategy Badge and Thinking Trace

Figure 4.4: AI Tutor — Chat Mode with Strategy Badge and Thinking Trace

Screenshot Placeholder

AI Tutor — Ask Course Mode with Citation Chips and Material Preview Panel

Figure 4.5: AI Tutor — Ask Course Mode with Citation Chips and Material Preview Panel

Screenshot Placeholder

AI Tutor — Quiz Mode with Animated Progress Ring and Beta Posterior Display

Figure 4.6: AI Tutor — Quiz Mode with Animated Progress Ring and Beta Posterior Display

Screenshot Placeholder

AI Tutor — Resource Discovery Mode with Type Filter Pills

Figure 4.7: AI Tutor — Resource Discovery Mode with Type Filter Pills

Screenshot Placeholder

Instructor Evaluation Dashboard (p95 latency, faithfulness, expertise curves)

Figure 4.8: Instructor Evaluation Dashboard (p95 latency, faithfulness, expertise curves)

Screenshot Placeholder

Analytics and Overview Dashboard (attendance, exam trends)

Figure 4.9: Analytics and Overview Dashboard (attendance, exam trends)

Screenshot Placeholder

Real-Time Notification Center

Figure 4.10: Real-Time Notification Center

Screenshot Placeholder

OCR Routine Scan — Editable Draft Before Confirmation

Figure 4.11: OCR Routine Scan — Editable Draft Before Confirmation

4.6 Performance and Functional Outcomes

Table 4.4: Observed Functional Outcomes During Validation Runs

Validation Scenario	**Outcome**
Authentication (login-refresh-logout)	Token issued, rotated, and invalidated correctly
RAG Ask-Course with uploaded PDF	Citations correct in 38/40 golden Q&A pairs
AI chat streaming response	Stable incremental SSE rendering, no dropped chunks
200 consecutive quiz generations	Zero JSON parse failures (vs. 6–9 previous)
Routine OCR upload	Editable draft presented before slot creation
Notification push on announcement	Socket event received within <100 ms
Material ingest (5-page PDF)	Worker completes in $\sim$12 s cold; status updated to DONE
Dark mode toggle	All pages switch without layout shift or FOUC

4.7 Grounded Retrieval-Augmented Q&A

The RAG implementation (Section 3.11) is the largest new deliverable. Four engineering decisions were load-bearing.

1. **Shared persistence.** Embeddings live in a `vector(768)` column on the same Postgres instance that stores user data, avoiding a second database service for capstone-scale workloads.

2. **Pagination-aware chunker.** The greedy packer targets 800 tokens with 150-token overlap and never crosses PDF page boundaries, which preserves page-number provenance

for citations.

3. **Hybrid fusion.** RRF with $K = 60$ fuses sparse and dense rankings without requiring calibrated scores — a practical win because cosine distance and `ts_rank_cd` are not directly comparable.

4. **Streaming citations.** SSE emits tokens from the generator and a final `event: citations` frame. The frontend `renderWithCitations` helper splices them into `[n]` tokens without re-rendering the text buffer, making citations feel instant.

4.8 Agentic Tool Use and Socratic Strategies

The chat controller delegates to the ReAct loop. Five strategies are implemented as prompt templates; a deterministic regex-based selector picks the strategy before generation starts, with a Socratic downgrade applied when the Beta-posterior mean falls below 0.4. Every strategy transition, tool call, and tool result is logged to `LlmCall` with a shared `parentCallId`, which is how the evaluation dashboard reconstructs an agent trace.

Structured quiz output is produced through Ollama's JSON-schema-constrained `format` parameter rather than post-hoc regex parsing. Across 200 consecutive generations during stability testing, zero parse failures were observed, down from an average of 6–9 failures per 200 with the previous regex approach.

4.9 Advanced OCR

The backend's `extractTextFromFile` function accepts a `quality` parameter. The fast path preserves the original Tesseract and `pdf-parse` behaviour. The accurate path calls the Python FastAPI sidecar (TrOCR for handwriting, Nougat for academic PDFs with embedded mathematics, Unstructured for layout-heavy documents) via `undici` with FormData. All accurate-path calls flow through a dedicated BullMQ worker so slow inferences never block HTTP handlers.

Availability-first default: if the sidecar returns non-200 or the underlying model fails to load, the backend falls back to the fast path and surfaces an "AI-transcribed — please verify" banner in the UI.

4.10 Bayesian Mastery Tracking

The conjugate Beta update described in Section 3.14 replaces the previous $+0.1/-0.05$ heuristic. The hyperparameters α and β are persisted on `TopicProgress`. After every quiz submission, the backend computes the posterior mean and 95% credible interval and

returns both to the client so the student can see their uncertainty explicitly. Adaptive difficulty uses the posterior mean to target the next quiz difficulty level.

4.11 LLM Observability

All Ollama callers flow through a `withLogging(route, fn)` higher-order function. Each call inserts a row into the `LlmCall` table capturing: route, model, latency, prompt and completion token counts, status (`OK`/`ERROR`/`TIMEOUT`), truncated error message, and `parentCallId` for agent trace reconstruction. Prometheus metrics are published at `/metrics` behind an `X-Metrics-Key` header: per-route latency histograms, token counters, and error rate gauges.

4.12 Quantitative Evaluation

The evaluation harness (`backend/src/eval/`) runs on a curated golden set of forty retrieval-augmented question-answer pairs spanning algorithms, distributed systems, machine learning, and operating systems. Metrics are computed by a three-times majority-vote LLM judge (faithfulness, context precision, answer relevance), a deterministic recall@K computation, and token-level WER/CER for OCR through the `jiwer` package.

Table 4.5 reports the headline numbers under the `qwen2.5:7b-instruct` + `nomic-embed-text` configuration.

Table 4.5: RAG quality and latency against the 40-question golden dataset.

Metric	**Value**	**Target**
Recall@5	0.83	≥ 0.80
Recall@8	0.88	—
Faithfulness (majority of 3)	0.79	≥ 0.75
Context precision	0.71	—
Answer keyword overlap	0.64	—
Cold answer latency (p50)	6.4 s	—
Cold answer latency (p95)	14.8 s	—
Warm answer latency (p50)	3.1 s	—

Table 4.6 reports OCR character and word error rates on a ten-sample golden set (five handwritten pages, five academic PDF pages with embedded equations).

Table 4.6: OCR CER and WER on the 10-sample golden OCR set (lower is better).

	Fast path (Tesseract / pdf-parse)		Accurate path (TrOCR / Nougat)	
Sample set	CER	WER	CER	WER
Handwritten (5 pages)	0.71	0.83	0.19	0.27
Academic PDF (5 pages)	0.28	0.41	0.07	0.12

Table 4.7: Agent and Quiz Stability Metrics

Metric	**Before upgrade**	**After upgrade**
Quiz JSON parse failure rate (per 200)	6–9	0
Strategy visible to student	No	Yes (badge in UI)
Mastery display	0.0–1.0 scalar	μ + 95% CI
LLM calls logged	0	100% (via `withLogging`)

4.13 Preliminary User Study

A within-subjects pre/post study was conducted with volunteer KUET CSE undergraduates over a two-week window following the protocol in `study/protocol.md`. Participants completed a ten-item pre-test on two topics, used the tutor freely for one week, then took a parallel ten-item post-test plus a System Usability Scale (SUS) [32, 33] survey and a one-item net promoter question.

The paired t-test on quiz accuracy and the SUS summary are produced by `scripts/analyze_study` Effect size is reported as Cohen's d on the differences. The fallback posture if recruitment yields fewer than six participants is documented in the protocol: preliminary self-case-study with the quantitative RAG and OCR metrics carrying the evaluative load. Full numerical tables and the pre/post boxplot are included in the award-submission packet when the study window closes.

4.14 Discussion

The upgraded implementation turns the previous generic LMS-plus-chat into a grounded, measurable, agent-driven tutor. Four outcomes stand out.

Architecture resilience. The route-controller-service separation inherited from the earlier iteration proved resilient: every new feature (RAG, sidecar OCR, evaluation harness, resource discovery) slots in behind an existing controller boundary without cross-cutting leaks.

Sidecar isolation. Moving all heavy Python ML into a FastAPI sidecar kept the backend Docker image small (<200 MB) and the cold-start time predictable. The fallback path means a sidecar crash does not break OCR entirely.

Zero-friction instrumentation. The `withLogging` wrapper instrumented every LLM call without touching any caller's surface API. The evaluation dashboard, Prometheus endpoint, and `LlmCall` table all come for free.

Interpretable mastery. Adopting conjugate Bayesian updates turned what was previously an opaque heuristic into a statement a student can read: "your mastery is 62%, 95% CI 47–76%." The strategy badge gives a second interpretability layer: the student can see that the tutor chose Socratic probing because their posterior mean was below 0.4.

The main residual risks are the structured-output reliability of Qwen2.5 7B on edge cases (mitigated with a 200-generation regression test), CPU-only Nougat latency on live demos (mitigated by pre-computing a demo sample cache), and the study recruitment risk (mitigated by an honest n=6 reporting fallback).

CHAPTER 5

Societal, Health, Safety, Legal, Cultural and Environmental Considerations

5.1 Societal Considerations

Cognitive Copilot addresses a concrete socioeconomic barrier: access to quality tutoring support. Private tuition is expensive and unavailable to many university students in Bangladesh. A local-first AI tutor that runs on department hardware costs nothing per query and is accessible to every enrolled student 24 hours a day. The grounded citation model further ensures that answers are tied to the course materials chosen by the faculty, not to generic internet text, which keeps the tutoring aligned with institutional learning objectives.

The community discussion and announcement modules promote peer-to-peer learning and reduce information asymmetry between instructors and students. Real-time Socket.IO notifications ensure that time-sensitive information (class cancellations, assignment deadlines) reaches students immediately rather than through delayed email chains.

5.2 Health and Safety Considerations

The platform itself introduces no direct physical hazard. However, two behavioural risks require acknowledgment.

1. **Screen time and sedentary study.** Extended device use is an established health concern. The system does not include usage limits, but institutional deployment guidelines should recommend scheduled breaks.

2. **Over-reliance on AI-generated answers.** A student who accepts the tutor's explanation without independent verification may develop superficial understanding. The "AI-transcribed — please verify" banner on sidecar OCR output, the Socratic strategy that withholds direct answers, and the citation model that forces reference back to source material all mitigate this risk by design.

5.3 Data Privacy and Security

The platform processes personally identifiable data (name, email, course enrollment, quiz performance history, and conversation logs). The following controls are in place.

Table 5.1: Security Controls and Compliance Notes

Risk	Control
Unauthorised account access	JWT (15 min) + refresh token rotation; bcrypt password hashing
Privilege escalation	Role-based middleware guard on all protected routes
Insecure file upload	MIME-type validation; uploads processed in worker, not HTTP thread
LLM conversation data leakage	All inference is local (Ollama); no data sent to cloud APIs
Sensitive metrics exposure	`/metrics` Prometheus endpoint gated by `X-Metrics-Key` header
SQL injection	All queries via Prisma parameterised ORM; raw SQL used only for vector ops
XSS via LLM output	Citation rendering uses React `dangerouslySetInnerHTML` only on sanitised text
Notification delivery failure	BullMQ exponential back-off retry (3 attempts)

Since all LLM inference runs locally through Ollama, student conversations never leave the institution's server. This is a deliberate design choice that eliminates the primary data-privacy risk of cloud-AI tutors.

5.4 Legal and Ethical Considerations

Academic integrity. The AI tutor is designed as a study aid, not as an answer generator for assessments. The Socratic strategy deliberately withholds answers to encourage independent reasoning. Institutional deployment should pair the tool with a clear academic-integrity policy statement.

Model bias. Qwen2.5 7B and nomic-embed-text are general-purpose models trained on primarily English text. Their outputs may reflect biases present in the training corpus. Faculty should review AI-generated explanations before endorsing them as canonical.

TrOCR Latin-only limitation. The handwriting OCR model does not support Bengali script, which is the dominant handwriting language for KUET students. This limitation is

documented prominently in the UI and in Chapter 7 of this report. Deployers must not present TrOCR output as reliable for Bengali handwriting.

LLM-as-judge evaluation bias. The faithfulness metric is scored by the same model family being evaluated. We report this limitation explicitly and use three-times majority voting to reduce variance.

5.5 Cultural Considerations

Language. The current interface is English-only. KUET students use both Bengali and English; future work should add Bengali UI labels and support Bengali academic PDFs through a Bengali-capable OCR model.

Date and time format. The system stores timestamps in UTC and allows per-user date format preference (ISO, US, BD) so that schedule display matches local convention.

Community norms. The community discussion module allows faculty to moderate thread content. Deployment guidelines should establish a code of conduct appropriate for academic discourse.

Diversity in learning styles. The five pedagogical strategies (Explain, Socratic, Hint Ladder, Worked Example, Misconception Probe) accommodate different learning preferences rather than applying a one-size-fits-all approach.

5.6 Environmental Considerations

Compared to cloud AI. Running inference locally eliminates network round-trips and cloud data-center energy usage for inference. The institution controls when and how the Ollama service runs, enabling scheduled shutdown during off-hours to reduce idle power consumption.

Local hardware cost. CPU-only inference on Qwen2.5 7B draws approximately 20–40 W of additional CPU power per concurrent user (measured on an AMD Ryzen 7 laptop). For a small departmental deployment of 20 concurrent users, this is comparable to a single CRT monitor and is environmentally negligible relative to the educational value delivered.

Model efficiency. Using a 7 B-parameter quantised model rather than GPT-4-class (100B+) models reduces per-query energy consumption by roughly two orders of magnitude at the cost of some capability ceiling. For the structured tasks in this system (quiz, citation, Socratic prompt), the 7B model is adequate and the energy-quality trade-off is favourable.

CHAPTER 6

Complex Engineering Problems and Activities

6.1 Overview

Engineering complexity arises when problems cannot be solved by straightforward application of standard knowledge and require judgment, analysis of trade-offs, and creative synthesis across multiple disciplines. This chapter maps the five most technically demanding sub-problems addressed in Cognitive Copilot against the criteria used in accreditation frameworks (Washington Accord / NBA) for complex engineering problems.

6.2 Complex Engineering Problems Addressed

6.2.1 Problem 1: Grounded, Cited Answer Generation via Hybrid Retrieval

Nature of complexity. Retrieval-augmented generation requires co-optimising two independently hard problems: information retrieval (sparse and dense) and constrained text generation. Neither problem is solvable in isolation: a retriever with poor recall produces cited-but-wrong answers; a generator unconstrained by retrieved context produces fluent hallucinations. Combining them within a latency budget on CPU-only hardware adds a third constraint.

Technical decisions made.

a) BM25 is implemented over Postgres `tsvector` rather than a dedicated search engine (Elasticsearch, Solr) to keep the deployment stack to a single database service. The trade-off is reduced BM25 flexibility (no custom analyzers), accepted because the vocabulary is academic English.

b) Cosine similarity is served by `pgvector` with HNSW indexing ($m = 16$, ef_construction $= 64$). These parameters balance recall (98% at 1000 vectors) against index build time on CPU-only hardware.

c) Reciprocal Rank Fusion with $K = 60$ combines the two ranking lists without requiring score calibration — a deliberate simplification whose correctness was verified against the 40-question golden dataset (recall@5 = 0.83, above the 0.80 target).

d) SSE streaming is used for the answer so the user sees tokens incrementally; a final citation frame delivers metadata without blocking the token stream.

Complexity criteria met. Involves conflicting constraints (latency vs. recall), requires non-standard knowledge (vector indexing, rank fusion theory), and demonstrates measurable performance improvement over the baseline (from zero grounded answers to recall@5 = 0.83).

6.2.2 Problem 2: Bounded Agentic Reasoning with Tool Use

Nature of complexity. An LLM agent must decide at each step whether to call a tool or produce an answer. This is a sequential decision problem with a non-deterministic state transition function (LLM output). Implementing it reliably requires: (a) a structured output format the LLM will not deviate from, (b) explicit iteration and wall-clock bounds to prevent runaway inference, (c) a fallback when the model produces malformed tool calls, and (d) logging of every step for audit.

Technical decisions made.

a) Ollama's `format` parameter constrains generation to a JSON schema defining the `thought`/`action`/`action_input` fields. This eliminates brittle post-hoc regex parsing.

b) The loop is capped at five iterations and twelve seconds wall-clock. On iteration five or timeout, the accumulated context is used directly for a final answer generation without a tool call.

c) Tool errors are caught and serialised as a `tool_result` entry; the model is instructed to adjust its plan rather than retry blindly.

d) Every loop step is written to `LlmCall` with the same `parentCallId`, enabling the instructor dashboard to reconstruct the full agent trace post-hoc.

Complexity criteria met. Involves depth of analysis (sequential reasoning with branching), non-deterministic components (LLM), and a need for systematic bounds to ensure safety and predictability.

6.2.3 Problem 3: Asynchronous ML Inference without Blocking HTTP

Nature of complexity. CPU-bound ML inference (nomic-embed-text embedding, TrOCR, Nougat) can take 2–30 seconds per call. If these calls block the HTTP event loop, the entire API becomes unresponsive for all users during a single upload. Node.js is single-threaded; the Python sidecar is multi-process but still CPU-bound. Coordinating asynchronous execution across two runtimes and keeping state consistent (ingest status, error reporting) is non-trivial.

Technical decisions made.

a) All heavy inference calls are moved to BullMQ workers (`ingest.worker.ts`, `ocr.worker.ts`) running in separate Node.js processes. The HTTP handler enqueues a job and returns immediately with a `202 Accepted` response.

b) Worker jobs write their progress to `Material.ingestStatus` (PENDING → PROCESSING → DONE/FAILED) so the frontend can poll or receive a Socket.IO notification on completion.

c) BullMQ retry policy uses exponential back-off (3 attempts, 5 s / 30 s / 120 s) so transient sidecar failures recover automatically.

d) The `ocrJobDuration` Prometheus histogram records sidecar round-trip time per quality tier, enabling capacity planning.

Complexity criteria met. Requires wide engineering knowledge (event loop, IPC, queue design, distributed state), involves conflicting constraints (throughput vs. latency), and requires systematic design rather than local fixes.

6.2.4 Problem 4: Structured LLM Output and JSON Schema Constraints

Nature of complexity. LLMs can generate arbitrary text, making it difficult to extract structured data reliably. The quiz generator requires a specific JSON schema: an array of five objects each with `question`, `options` (four strings), `correct` (one letter), and `explanation`. A single deviation (extra commentary, misplaced brace) causes a JSON parse error that crashes the quiz flow.

Solution and trade-offs. Ollama's `format` parameter accepts a JSON schema and constrains generation via grammar-based decoding. This gives zero parse failures across 200 test generations, verified in a stability test script. The trade-off is that grammar-constrained generation has slightly lower diversity than unconstrained generation; this is acceptable for a fixed-schema quiz format. The `chatCompletionStructured` function in `ollama.service.ts` encapsulates this behaviour for all structured calls.

Complexity criteria met. Requires identification and application of appropriate theoretical knowledge (constrained decoding), involves a risk of incorrect application, and demonstrates a measurable quality outcome (0 parse failures).

6.2.5 Problem 5: Calibrated Mastery Estimation with Minimal Data

Nature of complexity. Estimating a student's mastery from fewer than twenty quiz answers requires a principled statistical model that does not over- or under-fit. A point estimate with no uncertainty leads to premature difficulty targeting. A flat uniform distribution leads to no

adaptation. The model must also handle the cold-start case (zero answers) gracefully without defaulting to a misleadingly high or low mastery score.

Solution and trade-offs. The conjugate Beta posterior with uniform prior $\mathrm{Beta}(1, 1)$ gives a posterior mean of 0.5 and maximal variance at cold-start — interpretable as "we have no information yet, so we split difficulty equally." After ten correct out of twelve answers, the posterior is $\mathrm{Beta}(11, 3)$, mean 0.79, 95% CI $[0.55, 0.96]$, which narrows appropriately. This is simpler than Bayesian Knowledge Tracing (BKT) but produces wider credible intervals on sparse data, which is the honest behaviour for a small study. The complexity of DKT (Deep Knowledge Tracing) is unjustified for a per-student dataset of $O(10\text{–}100)$ attempts; Wilson et al. [20] verify this empirically.

Complexity criteria met. Requires probabilistic reasoning, involves a calibration trade-off (model complexity vs. data size), and produces a directly interpretable output displayed to the student.

6.3 Engineering Activities Performed

1. **Requirement decomposition.** User stories for all seven workstreams were decomposed into backend API contracts, Prisma schema changes, frontend component specifications, and acceptance tests before implementation began.

2. **Architecture design.** The layered route-controller-service-worker separation was established and documented with ADRs before any new code was written. Key decisions: single-database RAG (vs. separate vector store), sidecar for Python ML (vs. Python everywhere), BullMQ for all heavy jobs (vs. inline async/await).

3. **Database schema design.** The Prisma schema was extended with five new models (`Embedding`, `LlmCall`, `QuestionBank`, `StudySession`) and three new enums. The HNSW and GIN index parameters were chosen based on pgvector documentation and tested against a 10k-vector load.

4. **Iterative integration testing.** Frontend-backend integration was validated at each API boundary using Postman for streaming endpoints and the browser DevTools EventSource inspector for SSE frames.

5. **Automated golden-dataset evaluation.** A 40-question Q&A dataset with expected citations was curated and used to run reproducible recall@K and faithfulness metrics before each significant retrieval change.

6. **Stability testing.** The quiz generation stability test (`npm run test:quiz-stability`) runs 200 consecutive quiz generations and asserts zero parse failures, providing regression protection for the structured-output pipeline.

7. **Container and CI build.** Multi-stage Dockerfiles minimize image sizes; GitHub Actions CI runs lint, typecheck, Vitest unit tests, and an evaluation smoke subset on every pull request.

6.4 Technical Trade-Offs

Table 6.1: Key Engineering Trade-offs and Justifications

Decision	Trade-off accepted	Justification
pgvector in Postgres vs. dedicated vector DB	Limited HNSW tuning options	Single service; no extra infra
BM25 via tsvector vs. Elasticsearch	Less analyzer flexibility	No separate search service
qwen2.5:7b vs. larger model	Lower capability ceiling	Fits 8 GB RAM; runs on CPU
Sidecar for ML vs. Python monolith	Extra inter-process latency	Keeps Node image small; isolates deps
BullMQ workers vs. serverless functions	Worker process management	Works offline; no cloud needed
Conjugate Beta vs. BKT	Less complex state transitions	Appropriate for sparse data regime
JSON-schema constrained generation vs. regex	Slightly less diversity	Zero parse failures vs. 6–9 per 200

6.5 Outcome Against Engineering Objectives

All seven primary objectives stated in Chapter I were achieved. The resulting system is: (1) modular and extensible (new AI modes added without breaking existing endpoints), (2) measurable (evaluation harness produces reproducible numbers), (3) locally deployable (no cloud LLM, one-command Docker start), (4) observable (every LLM call logged, Prometheus endpoint active), and (5) pedagogically principled (Socratic strategies, Bayesian mastery, cited answers). The engineering investments in observability and evaluation give the project a base on which future capstone projects can build rather than a sealed artifact.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

Cognitive Copilot is a local-first, full-stack tutoring system that takes a deliberate position in a crowded field: every answer cites its source, every LLM call is measured, and every mastery claim is probabilistic rather than heuristic. The final deliverable combines (a) hybrid retrieval-augmented generation over pgvector, (b) transformer-based handwriting and academic-PDF OCR through a FastAPI sidecar, (c) a Socratic tool-using ReAct agent with five pedagogical strategies, (d) Bayesian Beta-posterior mastery tracking with 95% credible intervals surfaced in the UI, and (e) an end-to-end LLM observability and evaluation harness — all running on a single developer machine with no cloud LLM API.

From implementation and testing, the modular route-controller-service separation inherited from the earlier iteration scaled cleanly to the new subsystems. Structured JSON outputs eliminated the regex brittleness of the earlier quiz generator. The Beta posterior gave the mastery display a principled and interpretable form. The evaluation harness produced reproducible metrics (recall@5 ≈ 0.83, faithfulness ≈ 0.79 at the time of writing) that convert anecdotal claims about tutoring quality into auditable numbers. Taken together, the system achieves what we set out to do: engineer a tutoring experience that is *measurably* better than a generic chat, rather than merely differently styled.

7.2 Limitations

We report limitations honestly because over-claiming undermines the engineering.

1. **Small study sample.** The user study is underpowered for subgroup analyses. We present results as preliminary evidence, not as a clinical claim about adaptive-tutor effectiveness.

2. **English-only handwriting OCR.** TrOCR's pretrained checkpoint is Latin-only; Bengali handwriting recognition is not supported. Academic PDFs in English with embedded LaTeX are the only high-quality accurate-path case.

3. **CPU-only inference.** Qwen2.5 7B and nomic-embed-text run at acceptable latency on a modern laptop but remain sensitive to other workloads. Cold latency for a grounded answer is typically 6–15 seconds. A GPU would cut this substantially but is out of scope for the capstone.

4. **LLM-as-judge bias.** Faithfulness is scored by the same family of models under evaluation, which introduces correlation. We mitigate with three-times majority voting but do not claim the metric is unbiased.

5. **Chunk-boundary sensitivity.** The 800-token chunk size is a compromise; edge cases where a correct answer sits across a chunk boundary are under-served. Semantic chunking is listed as future work.

6. **No sandboxed network for `run_code`.** The agent code-execution tool runs in `node:vm` with a 2-second CPU limit and a 512 KB output cap, which is sufficient to prevent infinite loops but not adversarial escapes. We disable the tool by default outside trusted deployments.

7. **No multi-tenant isolation.** The system assumes a single institution deployment; embeddings, materials, and LLM logs share one Postgres instance. Multi-tenancy would require row-level security and per-tenant key management.

7.3 Future Work

1. **Bengali OCR.** Fine-tune TrOCR or use a Bengali-aware alternative to unlock the dominant handwriting use case for KUET students.

2. **Semantic chunking.** Move from fixed-token chunking to topic-segmentation-guided chunking; expected lift on recall@5 in preliminary offline experiments is 3–5 percentage points.

3. **Cross-encoder re-ranking.** The infrastructure for a cross-encoder re-ranking step is already gated behind `ENABLE_CROSS_ENCODER`; a systematic evaluation against recall-gain vs. latency cost is future work.

4. **Richer agentic tool set.** Add tools for spaced-repetition scheduling, LaTeX-to-rendered-image conversion, and course-wide concept-map generation.

5. **Mobile client.** A React Native front-end sharing the same backend would make the tutor usable on phones for the majority of KUET students.

6. **Larger user study.** Extend to two full course offerings ($n \geq 30$) with between-subjects A/B on Socratic vs. free-form chat modes, enabling causal claims rather than preliminary ones.

7. **GPU deployment guide.** Publish benchmarks and a Docker Compose overlay for CUDA-enabled Ollama so institutions with even a single modest GPU can cut latency by an order of magnitude.

7.4 Final Remarks

The system is suitable for academic demonstration and, with staged hardening, for a departmental-level deployment. More importantly, the engineering investments in observability, evaluation, and principled statistical models give the project a base on which future capstones can build rather than a sealed artifact. We release the code under an MIT licence in the hope that this helps.

Bibliography

[1] Meta Open Source, "React documentation," https://react.dev/, 2026.

[2] Express.js, "Express - node.js web application framework," https://expressjs.com/, 2026.

[3] PostgreSQL Global Development Group, "Postgresql documentation," https://www.postgresql.org/docs/, 2026.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[6] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[7] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," https://arxiv.org/abs/2402.01613, 2024.

[8] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[9] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *SIGIR '09: Proceedings of the 32nd International ACM SIGIR Conference*, 2009.

[10] A. Kane, "pgvector: Open-source vector similarity search for Postgres," https://github.com/pgvector/pgvector, 2024.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.

[12] Tesseract OCR, "Tesseract ocr user manual," https://tesseract-ocr.github.io/, 2026.

[13] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *AAAI Conference on Artificial Intelligence*, 2023.

[14] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," *arXiv preprint arXiv:2308.13418*, 2023.

[15] Unstructured AI, "Unstructured: Pre-processing tools for unstructured data," https://github.com/Unstructured-IO/unstructured, 2024.

[16] J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, "Cognitive tutors: Lessons learned," *The Journal of the Learning Sciences*, vol. 4, no. 2, pp. 167–207, 1995.

[17] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.

[18] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[19] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[20] K. H. Wilson, Y. Karklin, B. Han, and C. Ekanadham, "Estimating student proficiency: Deep learning is not the panacea," *arXiv preprint arXiv:1604.02416*, 2016.

[21] M. T. H. Chi, S. A. Siler, H. Jeong, T. Yamauchi, and R. G. Hausmann, "Learning from human tutoring," in *Cognitive Science*, 2001.

[22] Khan Academy, "Khanmigo: Khan academy's ai-powered learning guide," https://www.khanacademy.org/khan-labs, 2023.

[23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[24] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[25] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval-augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[26] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," *arXiv preprint arXiv:2311.09476*, 2023.

[27] Prisma, "Prisma orm documentation," https://www.prisma.io/docs, 2026.

[28] Redis Ltd., "Redis documentation," https://redis.io/docs/, 2026.

[29] Taskforce.sh, "Bullmq guide," https://docs.bullmq.io/, 2026.

[30] Ollama, "Ollama: Local large language model runtime," https://ollama.com/, 2026.

[31] Docker Inc., "Docker documentation," https://docs.docker.com/, 2026.

[32] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, 1996.

[33] A. Bangor, P. T. Kortum, and J. T. Miller, "An empirical evaluation of the system usability scale," *International Journal of Human-Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008.